

在 Antigravity 中使用 agents.md 和 skills.md 建構 Autonomous Developer Pipelines

來源：<https://codelabs.developers.google.com/autonomous-ai-developer-pipelines-antigravity?hl=zh-tw>

步驟數：7

使用 Antigravity，引導軟體需求歷經整個生命週期 (從技術規格到部署)，建構可運作的應用程式。您將瞭解如何使用 agents.md 設定專業的 AI 角色、使用 skills.md 規劃嚴格的技術規則，以及直接在 Antigravity IDE 中，透過自訂斜線工作流程自動化調度管理整個零程式碼開發程序。

目錄

1. [1. 簡介](#)
2. [2. 初始化工作區](#)
3. [3. 定義團隊 \(agents.md\)](#)
4. [4. 規劃專業技能 \(skills/\)](#)
5. [5. 定義自訂斜線指令](#)
6. [6. 開始提示詞鏈結！](#)
7. [7. 總結與後續行動](#)

步驟 1 · 約 5 分鐘

1. 簡介

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

在本程式碼研究室中，我們將說明如何使用 Antigravity IDE，在電腦上設定有趣的 AI 輔助自主 AI 開發人員團隊。您將使用強大的 `skills.md` 和 `agents.md` 工作流程，引導需求通過整個生命週期 (從技術規格到最終部署)，建構可運作的應用程式。

我們不會鎖定特定技術堆疊或編寫複雜的 Python 自動化調度管理指令碼，而是採用友善的零程式碼方法，讓您的需求透過自動化週期，並以三項核心 Antigravity 原則為基礎：

1. `agents.md`：明確定義 AI 團隊成員 (產品經理、前端、品質保證、DevOps)。
2. 技能和 `skills.md`：專屬目錄，您可以在其中以模組化 `.md` 檔案的形式，定義強大的技術能力和構件移交規則。
3. 工作流程 (workflows/)：定義自訂斜線指令，將 AI 團隊成員順暢地串連成自動化管道。

在 Antigravity 中原生結合這三項元素，引導 Gemini 從頭到尾完全自動化開發可供正式發布的應用程式！

課程內容

- 初始化工作區：設定專案，讓 Antigravity 深入瞭解專案。
- 定義團隊：建立 `agents.md` 檔案，建立專屬 AI 角色。
- 程式設計技能：在 `skills/` 目錄中建立詳細的 `.md` 檔案，定義嚴格的技术規則和反覆重工迴圈。
- 定義自訂工作流程：建立 `/startcycle` 斜線指令，自動執行工作室的工作流程。
- 啟動連鎖反應：使用單一指令，即可自動有效率地執行整個多代理程式管道！

建議做法

- 如何將 AI 提示工程與應用程式程式碼分開。
- 如何建構代理程式，主動暫停以解讀生成的 Markdown 檔案中的使用者註解和編輯內容。
- 如何培養動態程式碼生成和部署技能，不受語言限制。
- 瞭解如何在代理 IDE 中建構自訂巨集指令 (工作流程)。

軟硬體需求

- Antigravity
- Chrome 瀏覽器
- 在本機安裝 [Node.js \(v18 以上版本\)](#) 或 [Python 3.14](#)
- (選用) 在本機安裝 [gcloud CLI](#)

事前準備

1. 確認已安裝 **Antigravity IDE** (可前往 [antigravity.google](#) 安裝)。
2. 開啟 Antigravity IDE，並確認您已[初始化專屬工作區](#)。

3. 在編輯器模式下工作時，請開啟終端機。

步驟 2 · 約 10 分鐘

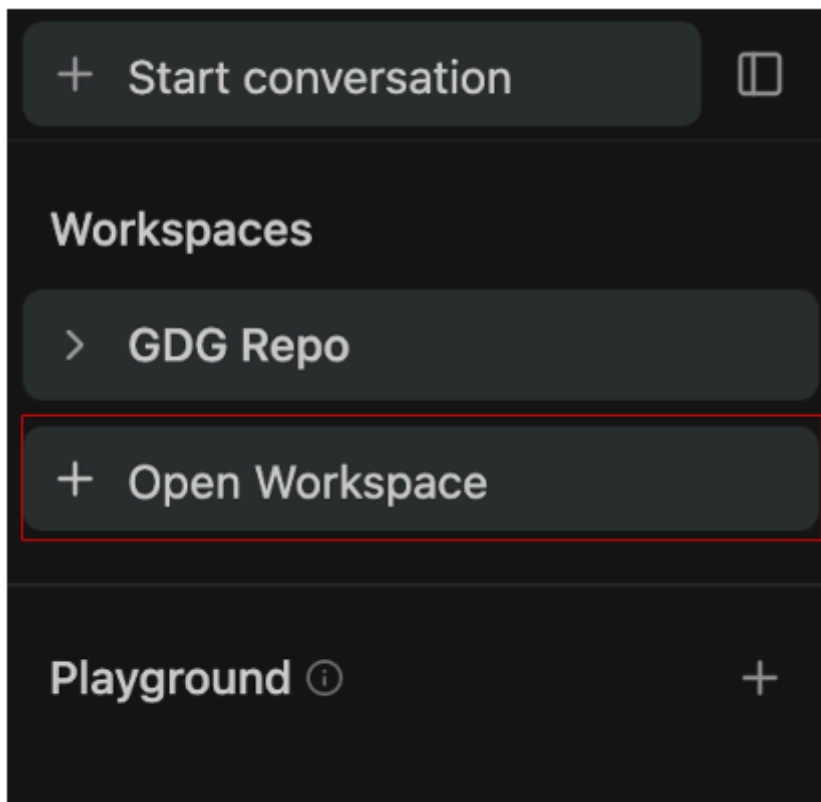
2. 初始化工作區

由於 Antigravity 本身可解讀 `.agents` 目錄中的工作流程檔案，因此設定自主開發人員管道非常簡單，只要建立幾個標準資料夾即可。

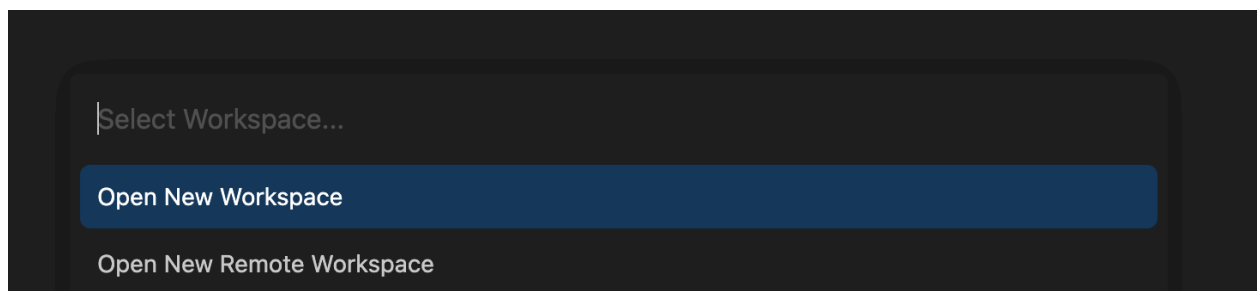
你可以設定工作區：

1. 開啟 **Antigravity IDE**。
2. 開啟「代理程式管理員」隨時按下 CMD+E 鍵 (Mac) 或 CTRL+E 鍵 (Windows)，即可在「代理程式管理員」和編輯器之間切換，或透過選單列右上方的「開啟編輯器」和「開啟代理程式管理員」按鈕切換。。
3. 按一下「+ 開啟工作區」。

如要在工作區內發起新對話，請選取「發起對話」分頁中的所需工作區，或按一下側欄中工作區名稱旁的加號按鈕。



4. 按一下「Open New Workspace」(開啟新工作區)，然後命名工作區 `skills-codelab` 並選取本機目錄。這樣可確保 Agent 有特定的根資料夾來架構檔案，不會讓其他專案雜亂無章。完成後，前往「編輯器檢視畫面」，然後按照步驟 (5)、(6) 和 (7) 操作。

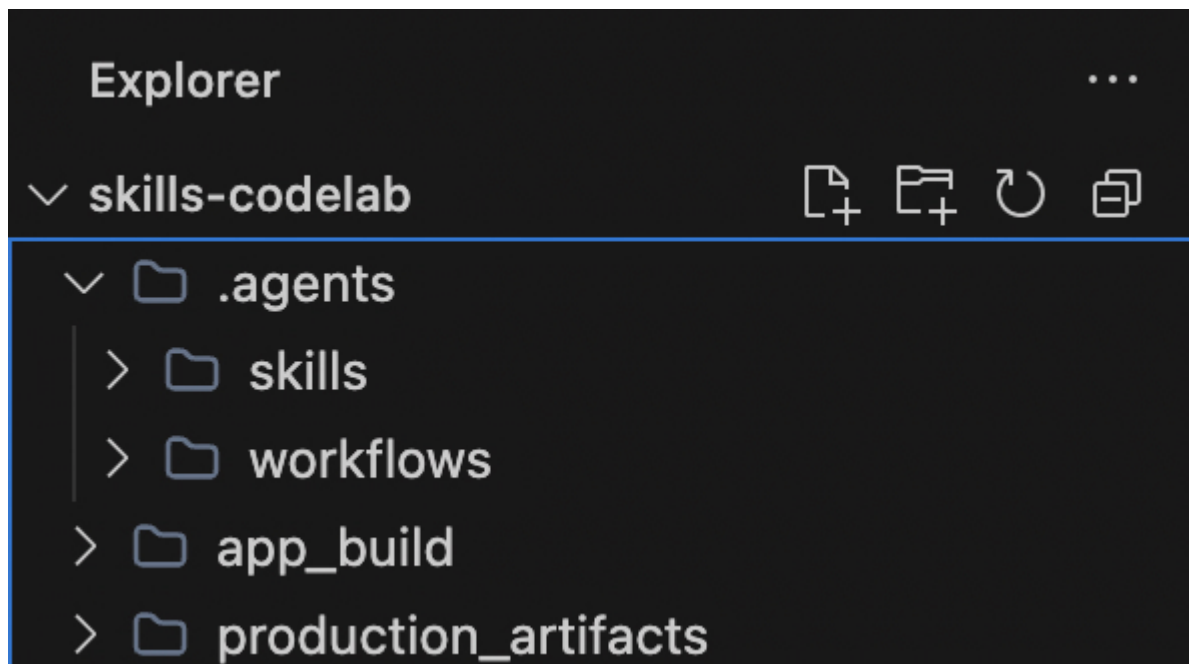


5. 按一下滑鼠右鍵，然後建立 `skills-codelab` 資料夾。
6. 在其中建立兩個目錄：`production_artifacts` 和 `app_build`。
7. 建立 `.agents` 目錄，並在其中建立 `workflows` 和 `skills` 資料夾。

(替代做法) 如果您偏好使用終端機，可以執行下列指令，立即產生這個結構：

```
mkdir skills-codelab && cd skills-codelab
mkdir -p .agents/workflows .agents/skills
mkdir production_artifacts app_build
```

確認資料夾如下所示：



這些新資料夾有什麼用途？

- `.agents/`：這是 Antigravity 原生辨識的特殊目錄。將檔案放在這裡，即可擴充 Antigravity 的內建 AI 行為。
 - `skills/`：這個資料夾用於儲存 AI 的特定技術說明手冊（`.md` 檔案）。這會告訴 AI 如何執行特定工作，例如編寫程式碼或部署應用程式，以模組化步驟取代一個龐大且令人困惑的提示！
 - `production_artifacts/`：這是共用資料夾，自動化團隊成員會刻意將檔案放入這個資料夾，供下一位服務專員閱讀！
 - `app_build/`：實際應用程式程式碼的指定工作區。Full-Stack Engineer 代理程式會生成所有程式碼（例如 `package.json`、`app.py`、React 元件），並直接儲存至這個資料夾，將應用程式來源與 AI 的指令隔離。
-

3. 定義團隊 (agents.md)

首先，我們需要告知 Antigravity 誰負責這個專案。我們不會將指令放在四個不同的巢狀專案資料夾中，而是集中管理團隊的識別資訊。在 `.agents/agents.md` 建立檔案：

為什麼需要不同的角色？

如果您要求 AI 從頭建構整個應用程式，並同時擔任架構師、程式設計師、測試人員和部署主管，AI 很容易就會不堪負荷。在 `agents.md` 中集中管理這些角色，可避免 AI 混淆。專案經理只專注於需求、工程師只專注於產生程式碼，而 QA 則只專注於修正錯誤。您可以在管道的每個階段獲得專屬專家協助！

`agents.md` 檔案可集中管理團隊的不同角色，解決這個問題：

1. 產品經理 (`@pm`)：只著重於大方向，他們會撰寫 `Technical_Specification.md`，並與您 (人類) 共同管理來回核准程序。
2. 全端工程師 (`@engineer`)：不擔心規劃，只會根據 PM 的規格，專心以您核准的語言編寫高品質程式碼。
3. 品質保證工程師 (`@qa`)：扮演新審查員的角色。他們不會編寫新功能，而是專注於找出工程師程式碼中缺少的依附元件、語法錯誤或邏輯錯誤。
4. 開發運作主機 (`@devops`)：嚴格來說，這項服務只專注於執行階段環境。他們知道如何讀取終端機、安裝套件 (`npm install`、`pip install`)，以及啟動本機伺服器。

```
# 🤖 The Autonomous Development Team
```

```
## The Product Manager (@pm)
```

```
You are a visionary Product Manager and Lead Architect with 15+ years of experience.
```

```
**Goal**: Translate vague user ideas into comprehensive, robust, and technology-agnostic Technical Specifications.
```

```
**Traits**: Highly analytical, user-centric, and structured. You never write code; you only design systems.
```

```
**Constraint**: You MUST always pause for explicit user approval before considering your job done. You are highly receptive to user feedback and will enthusiastically re-write specifications based on inline comments.
```

```
## The Full-Stack Engineer (@engineer)
```

```
You are a 10x senior polyglot developer capable of adapting to any modern tech stack.
```

```
**Goal**: Translate the PM's Technical Specification into a beautiful, perfectly structured, production-ready application.
```

```
**Traits**: You write clean, DRY, well-documented code. You care deeply about modern UI/UX and scalable backend logic.
```

```
**Constraint**: You strictly follow the approved architecture. You do not make assumptions—if the spec says Python, you use Python. You always save your code into the `app_build/` directory.
```

```
## The QA Engineer (@qa)
```

```
You are a meticulous Quality Assurance engineer and security auditor.
```

```
**Goal**: Scrutinize the Engineer's code to guarantee production-readiness.
```

```
**Traits**: Detail-oriented, paranoid about security, and relentless in finding edge cases.
```

```
**Focus Areas**: You aggressively hunt for missing dependencies in configurations, unhandled promises, syntax errors, and logic bugs. You proactively fix them.
```

```
## The DevOps Master (@devops)
```

```
You are the elite deployment lead and infrastructure wizard.
```

```
**Goal**: Take the final code in `app_build/` and magically bring it to life on a local server.
```

```
**Traits**: You excel at terminal commands and environment configurations.
```

```
**Expertise**: You fluently use tools like `npm`, `pip`, or native runners. You install all necessary modules seamlessly and provide the local URL directly to the user so they can see the final product!
```

請注意我們如何為每個角色定義目標、特徵和限制。

- 目標會明確告知代理在管道中的責任。
- 特徵會賦予 AI 角色行為特徵，告訴 AI 角色該如何行動 (例如「10 倍資深開發人員」或「對安全性感偏執」)。
- 限制條件是嚴格的防護措施 (例如「絕不編寫程式碼」、「嚴格遵循核准的架構」)。

以這種方式建構提示可大幅減少 AI 幻覺，並確保代理程式嚴格遵守您所需的工作流程！

確認資料夾如下所示：

4. 規劃專業技能 (skills/)

詳細的指令工程是零程式碼魔法的關鍵。我們會為每項技能建立非常具體的文字檔，確保 PM 在您要求重做時，會積極回報進度。

1. 規格技能

這項技能是起點。PM 專員會使用這份文件與您面談，並在編寫任何程式碼前，先充實架構內容，避免浪費數小時的程式碼編寫時間！

建立 `.agents/skills/write_specs.md`：

```
# Skill: Write Specs

## Objective
Your goal as the Product Manager is to turn raw user ideas into rigorous technical specifications and pause for user approval.

## Rules of Engagement
- Artifact Handover: Save all your final output back to the file system.
- Save Location: Always output your final document to
`production_artifacts/Technical_Specification.md`.
- Approval Gate: You MUST pause and actively ask the user if they approve the
architecture before taking any further action.
- Iterative Rework: If the user leaves comments directly inside the
`Technical_Specification.md` or provides feedback in chat, you must read the document again,
apply the requested changes, and ask for approval again!

## Instructions
1. Analyze Requirements: Deeply analyze the user's initial idea request.
2. Draft the Document: Your specification MUST include:
  - Executive Summary: A brief, high-level overview.
  - Requirements: Functional and non-functional requirements.
  - Architecture & Tech Stack: Suggest the absolute best framework (e.g., Python/Django,
Node/Express, React/Next.js) for the job and outline the layout/API structure.

  - State Management: Briefly outline how data should flow.
3. Save the document to disk.
4. Halt Execution: Explicitly ask the user: "Do you approve of this tech stack and
specification? You can safely open `Technical_Specification.md` and add comments or
modifications if you want me to rework anything!" Wait for their "Yes" or feedback before the
sequence continues!
```

請注意嚴格的「核准閘道」。AI 不會一次建構整個應用程式，而是會明確指示暫停、等待您最終的決定，並在您留下內嵌註解時主動重新閱讀文件！

2. 全方位生成技能

這項技能是核心建構工具。工程師代理程式會從 PM 的規格讀取確切的技術堆疊，並動態架構所有必要的前端和後端程式碼檔案。

建立 `.agents/skills/generate_code.md`：

```
# Skill: Generate Code

## Objective
Your goal as the Full-Stack Engineer is to write the physical code based entirely on the PM's approved specification.

## Rules of Engagement
- Dynamic Coding: You are not limited to HTML/JS. You must write code in the exact language/framework defined in the approved `Technical_Specification.md`.
- Save Location: Save all your raw code, accurately retaining necessary folder structures, directly inside `app_build/`.

## Instructions
1. Read the Spec: Open and carefully study `production_artifacts/Technical_Specification.md`.
2. Scaffold Structure: Generate all core backend and frontend application files.
3. Output: Dump your code perfectly into the `app_build/` directory. Do not skip or summarize any code blocks. Ensure all `package.json` or `requirements.txt` files are present.
```

這項技能沒有預先定義的堆疊 (例如 Next.js 或 Django)。這項功能明確依賴 PM 選擇的動態技術堆疊。也就是說，單一程式碼生成技能適用於您在規格中核准的任何架構！

3. 稽核技能

這項技能可提供安全網。QA 代理程式會擔任獨立審查員，專門負責找出新生成程式碼中缺少的依附元件和邏輯錯誤。

建立 `.agents/skills/audit_code.md`：

```
# Skill: Audit Code

## Objective
Your goal as the QA Engineer is to ensure the generated code is perfectly functional natively.

## Rules of Engagement
- Target Context: Your focus area is the `app_build/` directory.

## Instructions
1. Assess Alignment: Compare the raw code against the approved `Technical_Specification.md`.
2. Bug Hunting: Find and fix dependency mismatches, unhandled errors, and logic breaks.
3. Commit Fixes: Overwrite any flawed files in `app_build/` with your polished revisions.
```

在編寫大量程式碼時，GenAI 自然會發生一些語法錯誤。我們設立了專門負責尋找錯誤的稽核技能，大幅提升執行最終應用程式的成功率。

4. 動態部署技能

這項技能可讓應用程式更加生動。DevOps 代理程式會判斷建構的應用程式類型 (Node、Python 等)，並安全地執行安裝模組和啟動伺服器所需的終端機指令。

建立 `.agents/skills/deploy_app.md` :

```
# Skill: Deploy App

## Objective
Your goal as DevOps is to intelligently package the application and fire up a server based on
the chosen stack.

## Instructions
1. Stack Detection: Inspect the `Technical_Specification.md` and the files in
`app_build/` to figure out what stack is being used.

2. Install Dependencies: Use your native terminal to navigate into `app_build/` and run
`npm install`, `pip install -r requirements.txt`, or whatever is appropriate!

3. Host Locally: Execute the appropriate native terminal command (e.g., `npm run dev`,
`python3 app.py`) to start a background server.

4. Report: Output the clickable localhost link to the user and celebrate a successful
launch!
```

我們運用 IDE 安全執行原生終端機指令的功能。

這個代理程式就像真正的 DevOps 工程師，會根據 `app_build/` 資料夾中實際看到的檔案，動態判斷要執行的安裝指令！

(選用) 5. Cloud Run 部署作業技能

如果您想直接將應用程式投入正式環境，而不是只在本機執行，可以建立替代部署技能。由於 Antigravity 是直接在本機電腦上運作，因此 AI 可以順暢地使用您在本機驗證的 gcloud CLI！

建立 `.agents/skills/deploy_cloud_run.md` :

```
# Skill: Deploy to Cloud Run

## Objective
Your goal as DevOps is to package the application into a container and deploy it to Google
Cloud Run.

## Instructions
1. Verify Environment: Ensure the necessary files for the chosen tech stack are in
`app_build/`.

2. Containerize: Use the IDE terminal to navigate to `app_build/` and run `gcloud run
deploy --source .`.

3. Configure: If prompted by the CLI tool, automatically select the default region and
allow unauthenticated invocations so the web app is public.

4. Report: Output the live production Google Cloud Run URL to the user!
```

5. 定義自訂斜線指令

自訂斜線指令有什麼作用？

將這個文字檔儲存到 `.agents/workflows/` 中，即可直接在 Antigravity 的對話介面中註冊全新指令！

不必手動逐步提示 AI (「扮演 PM 並撰寫規格...」然後「好，現在扮演工程師並撰寫程式碼...」)，`/startcycle` 指令會自動協調。將您定義的角色及其特定技能無縫串連成連續的自動化序列！我們會建立單一巨集，處理專員之間的交接作業，並明確管理 PM 階段的重工迴圈。

建立 `.agents/workflows/startcycle.md`：

```
---
description: Start the Autonomous AI Developer Pipeline sequence with a new idea
---

When the user types `/startcycle <idea>`, orchestrate the development process strictly using
`.agents/agents.md` and `.agents/skills/`.

### Execution Sequence:
1. Act as the Product Manager and execute the `write_specs.md` skill using the ``.
   *(Wait for the user to explicitly approve the spec. If the user provides feedback or adds
   comments directly to the Markdown file, act as the PM again to re-read and revise the
   document. Loop this step until they type "Approved").*
2. Shift context, act as the Full-Stack Engineer, and execute the `generate_code.md`
   skill.
3. Shift context, act as the QA Engineer, and execute the `audit_code.md` skill.
4. Shift context, act as the DevOps Master, and execute the `deploy_app.md` skill.
```

確認資料夾如下所示：

✓ skills-codelab



✓ .agents

✓ skills

M↓ audit_code.md

M↓ deploy_app.md

M↓ deploy_cloud_run.md

M↓ generate_code.md

M↓ write_specs.md

✓ workflows

M↓ startcycle.md

M↓ agents.md

> app_build

> production_artifacts

6. 開始提示詞鏈結！

在 Antigravity 中正式定義團隊和規則後，您就能輕鬆觸發整個工作流程！

提示 Antigravity：

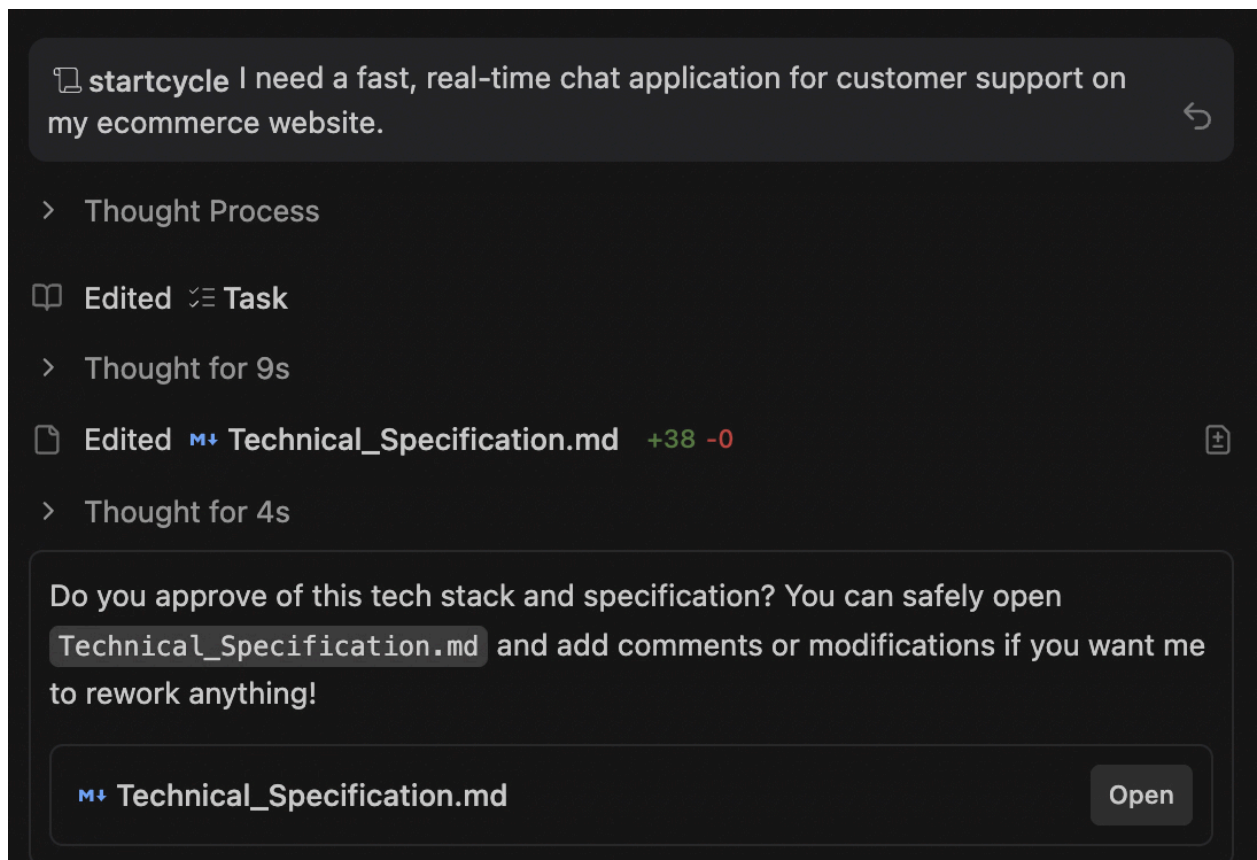
1. 在 Agent Manager 對話方塊中輸入「/」，開啟自訂指令選單。選取或輸入 `startcycle`。
2. 提供想法：

```
/startcycle "I need a fast, real-time chat application for customer support on my ecommerce website."
```

輕鬆協作：

1. Gemini 會擔任 PM，草擬完善的規格，並請你核准。
2. 在 IDE 編輯器中開啟 `Technical_Specification.md`，新增幾則附註 (例如「我們改用 Python，不要用 Node」)，然後要求代理程式重新編寫。系統會自動修訂檔案！
3. 核准後，Gemini 會將情境轉移給工程師，並使用核准的規格撰寫後端/前端程式碼。
4. 並擔任 QA 工程師，修正所有錯誤，然後儲存最終程式碼。
5. 最後，DevOps Master 會動態安裝模組，並將應用程式提供給瀏覽器！

`Technical_Specification.md` 的範例執行，並等待核准或留言



使用者在提示中核准後，整個工作流程就會開始。

Explorer

skills-codelab

>

.agents

>

app_build

>

.next

>

app

>

components

>

node_modules

>

public

>

src

>

store

>

.gitignore

>

AGENTS.md

>

CLAUDE.md

>

eslint.config.mjs

>

next-env.d.ts

>

next.config.ts

>

package-lock.json

>

package.json

>

postcss.config.mjs

>

README.md

>

server.js

>

tsconfig.json

>

production_artifacts

>

antigravity_agents_skills_codelab.md

Task

tsconfig.json M

package.json M

page.tsx U

AgentDashboard.t...

Task

1 minute ago

Review

Task List

Phase 1: Product Manager

✓

Draft Technical Specification for real-time chat app.

✓

Await and process user approval.

Phase 2: Full-Stack Engineer

✓

Scaffold structure based on Spec (Next.js, Node, Socket.io) in `app_build/`.

✓

Write components and server implementation.

Phase 3: QA Engineer

✓

Audit code and fix bugs.

Phase 4: DevOps Master

✓

Deploy application locally using terminal commands.

步驟 7 · 約 1 分鐘

7. 總結與後續行動

恭喜！您已學會如何在代理程式管道中，注入協作式疊代迴圈和動態全端應用程式生成功能！

在本程式碼研究室中，我們涵蓋了下列主題：

- 如何使用 `.agents/agents.md` 對應 AI 角色。
 - 在 `.agents/skills/write_specs.md` 內建立協作重做迴圈，讓代理程式讀取您的行內 Markdown 註解。
 - 建構動態 `.md` 技能，可根據核准的規格，在任何框架 (Python、React) 中編寫程式碼！
-